

How Kreaton was built and deployed

Every part of the system, what it is made of, why that choice over the alternatives, and how it got from an empty folder to a public URL with a green pipeline. Written so that a reader can check each claim against the repository rather than take it on trust.

Repository [junekashif/Kreaton](#) (public) · submission copy [Cognitia-IEM/KREATON](#)

Live [kreaton-upi.vercel.app](#) · **Engine** 1.0.0-kreaton-2026-4000x60 · **Tests** 107 · **Gate** 16 checks

Figures in this document are from the committed artefacts as of 12 September 2026.

Contents

- | | |
|--|--|
| 1. The problem, in one page | 8. The console — apps/web |
| 2. Architecture at a glance | 9. The API |
| 3. The engine — packages/core | 10. Persistence — where state lives |
| 4. The model — how it was trained and what it scores | 11. Testing and the quality gate |
| 5. Data — the corpus, and what it is not | 12. Deployment — GitHub Actions to Vercel |
| 6. Simulation, fitting and evaluation — packages/sim | 13. Things that went wrong, and what they taught |
| 7. Bringing in outside data — packages/ingest | 14. What is not built |

1. The problem, in one page

Authorised push payment (APP) fraud is the case where the victim is present, authenticated, and pressing the button themselves. Device fingerprinting, credential checks and velocity rules built for account takeover see nothing wrong, because nothing about the *authentication* is wrong. What is wrong is the *intent*, and intent leaves a different fingerprint: the beneficiary is new and was added minutes ago by pasting an identifier sent over chat; the payer is on a call and moving faster than usual; the amount is far outside their history; the receiving account is young, receives from many unrelated payers and empties within minutes.

Kreaton is a sending-side, authorisation-time control. It sees the payment before settlement, has a budget of a few milliseconds, and can approve, hold for re-confirmation, or decline. Two framing choices shape everything downstream:

- **Money is the objective, not a score.** The system never compares a risk score to a threshold somebody picked. It computes the expected cost, in rupees, of each available action and takes the cheapest. The threshold falls out of the arithmetic and moves with the amount.
- **Every decision must be reconstructible by hand.** The score is a sum. The contributions are listed. The costs are listed. The audit record contains all of it, sealed. A reviewer who disagrees can point at the line that produced the decision.

2. Architecture at a glance

kreaton/	npm workspaces monorepo, TypeScript throughout
├ packages/core	the engine: zero runtime dependencies, isomorphic
├ packages/ingest	dataset import: delimited/JSON → engine transactions
├ packages/sheets	durable write-behind sink to Google Sheets
├ packages/sim	corpus generator, fitter, replay, metrics, adversary, CLI
├ apps/web	Next.js console and API routes
├ analysis/	Python cross-check of the statistics (offline only)
├ data/	committed artefacts: fitted model, metrics, reports
├ docs/	MODELING, MODEL_CARD, SENSITIVITY, DATA_INPUT, PERSISTENCE
└ .github/workflows	ci.yml and deploy.yml

LAYER	TECHNOLOGY	LINES	WHY
packages/core	TypeScript 5.7, no dependencies	4,671	Runs identically in a browser tab and a serverless function. Nothing to audit but our own code.
packages/sim	TypeScript, <code>tsx</code> for scripts	6,050	Everything offline: generation, fitting, evaluation, adversarial suite, CLIs.
packages/ingest	TypeScript, no dependencies	1,757	Same importer in the browser, the API and the CLI.
packages/sheets	TypeScript + <code>node:crypto</code>	650	Service-account JWT signed directly; no <code>googleapis</code> tree.
apps/web	Next.js 16.3, React 19.2, App Router, plain CSS	8,604	No UI library, no chart library. Charts are hand-drawn SVG.
analysis/	Python 3.11, NumPy, SciPy, scikit-learn, pandas	442	Reproduces the TypeScript statistics independently. Nothing at runtime reads it.
Tests	Vitest 2.1	1,233	107 tests across four packages.

The unusual choice is that **the console runs the real engine client-side**. Because `packages/core` has no dependencies and an authorisation takes tens of microseconds, the browser tab executes the same compiled scorer as the server, against a pre-warmed slice of the corpus. What you see on screen is not a recording of results; it is the scorer running.

3. The engine — packages/core

One payment enters `Interceptor.authorize()` and leaves with an `Assessment`: every signal's measurement and contribution, the fused log-odds, the calibrated probability, the expected cost of each action, the recoverability estimate, the decision, any hold opened, and the sealed ledger position. The steps, in order:

1. **Extract** twelve raw statistics from the payment and the payer's and payee's profiles (`signals/extract.ts`).
2. **Bin** each against declared edges and look up its fitted log-likelihood ratio (`signals/specs.ts`, `fusion.ts`).
3. **Weight, shrink, cap and sum** into log-odds; verify additivity (`fusion.ts`).
4. **Calibrate** to a probability through the isotonic map (`fusion.ts`).
5. **Estimate recoverability** — what share of the money could still be frozen at the expected report lag (`recovery.ts`).
6. **Price** approve, hold and block in pause; take the cheapest (`cost.ts`, `policy.ts`).
7. **Open or link a hold** if the answer is hold, choosing a challenge the coach on the phone cannot relay (`hold.ts`).
8. **Seal** the assessment into the hash-chained ledger (`ledger.ts`).
9. **Update** the payer and payee profiles — last, so the payment is never scored against a baseline that already contains it (`profiles.ts`, `engine.ts`).

The twelve signals

SIGNAL	GROUP	WHAT IT MEASURES	WEIGHT
Beneficiary novelty	payee_graph	Hours since this payer first paid this beneficiary	0.779
Amount deviation	behavioural	Robust z-score of log amount against the payer's own history (median, MAD)	1.160
Time-of-day surprisal	behavioural	$-\log P(\text{hour})$ under the payer's own hour histogram	1.072
Device trust age	identity_device	Hours since the device was bound; zero on a SIM swap	0.972
Unrelated inflow that leaves immediately	payee_graph	Distinct payers in 24h $\times$ fraction forwarded within the hour	1.253
Beneficiary account age	payee_graph	Hours since the account was first seen anywhere in the network	0.879
Concurrent coercion indicators	context	Weighted count: call duration, screen share, remote-access app	0.908
Session urgency	context	App switches, entry method, seconds from open to authorise	0.831
Threshold structuring	structuring	Placement below limits, window totals crossing them, split re-attempts	0.590

SIGNAL	GROUP	WHAT IT MEASURES	WEIGHT
Velocity burst	behavioural	Payments in 24h against the payer's daily baseline	0.826
Account drain ratio	behavioural	Amount over available balance — observed if known, else estimated	0.135
Graph proximity to a confirmed mule	payee_graph	Hops to the nearest confirmed collection account	1.237

Choices worth defending

Weight-of-evidence binning over raw features. Each bin carries a Laplace-smoothed $\ln \frac{P(\text{bin}|\text{fraud})}{P(\text{bin}|\text{legit})}$. This is the classic credit-scoring form. It handles non-monotone signals without a transform, makes every contribution a number that sums exactly to the score, and gives every fired bin a stable reason code (`APP-AMT-7` , `APP-CAL-4`) that a regulator can be given a table for.

Additive log-odds, not a tree ensemble or a neural network. Three reasons. It is an explanation, not a summary: the engine asserts on every decision that the contributions sum to the score, and throws if they do not. It is fast enough to run in a browser tab. And its failure modes are visible in the table, where a boosted model that has learned an artefact of the generator is wrong invisibly.

Group correlation shrinkage. Four behavioural signals firing together are not four independent pieces of evidence. Intra-group correlation is estimated on the training window and each of k contributing signals in a group is discounted by $1/(1+(k-1)\rho)$. Applied inside the design matrix during fitting, so the weights are fitted against exactly what the runtime computes.

The asymmetric evidence cap. A fitted model learns that scams almost always coincide with a call, urgency and a pasted identifier — and therefore that the *absence* of those is strong evidence of legitimacy. An attacker controls those indicators. The cap floors the total exculpatory contribution from attacker-controllable signals, so a "hygienic" attacker cannot buy their way to approval by keeping the victim off the phone. This is a security decision, not a statistical one, and the adversarial suite measures what it costs and what it buys.

Robust statistics for the personal baseline. Median and MAD in log space, not mean and standard deviation. MAD has a fifty percent breakdown point: an attacker who seeds the baseline with one large payment before the extraction — a real attack, and one the suite runs — cannot shift it.

Expected cost, not a threshold. Each action is priced as liability (probability $\times$ amount $\times$ unrecovered share $\times$ liability share) plus friction (handling, abandonment, lifetime value at risk). The cheapest wins. The boundary therefore moves with the amount, and the `/policy` page lets a risk officer move it by changing prices rather than by guessing a cut-off.

4. The model — how it was trained and what it scores

0.9960

held-out ROC AUC

0.8895

PR AUC at 0.30% prevalence

0.0019

expected calibration error

93.4%

recall at 1% FPR

Training data

Corpus	4,78,090 payments over 60 days for 4,000 payers, 700 merchants, 60 mule chains; seed <code>kreaton-2026</code>
Fraud	1,434 (0.30%), across eight typologies
Split	Chronological at day 36 — 2,86,854 train / 1,91,236 test. Never random.
Fraud in training	421 payments

Fitting pipeline

1. Generate the corpus deterministically from the seed (`generator.ts` , `typologies.ts`).
2. Replay it chronologically through the engine with an unfitted model, collecting every raw signal (`replay.ts`). Beneficiary intelligence is released on its own delayed timeline so no payment is scored with knowledge that arrived after it.
3. On the training window, compute per-bin weights of evidence with Laplace smoothing; estimate intra-group correlations (`fit.ts`).
4. Fit per-signal weights by iteratively reweighted least squares with a ridge penalty centred on **one**, not zero — the null model is "trust the tables", and the penalty pulls a weight away only when the data insist.
5. Fit the calibration map by isotonic regression (pool-adjacent-violators) on the training window.
6. Evaluate on the test window; write `model.json` , `fit.json` , `metrics.json` .

Operating point under the default policy

MEASURE	VALUE	NOTE
Detection by count	80.9%	820 of 1,013 held-out scams intervened
Detection by value	95.7%	the large ones are caught
False-positive rate	0.145%	276 of 190,223 genuine payments interrupted
Precision	74.8%	three in four interventions were scams
Liability avoided	₹8.90 Cr	net of expected recovery
Friction cost	₹3.3 L	holds and blocks on genuine payments
Net benefit	₹8.86 Cr	on the 40-day test window

MEASURE	VALUE	NOTE
Latency p99	0.118 ms	mean 0.039 ms

Adversarial evaluation

Six attack strategies — structuring, baseline building, delayed extraction, context hygiene, threshold probing, mule rotation — each run for 400 episodes against three defences tuned to the same friction so the comparison is fair. Under the balanced policy the expected-cost engine catches **1,772 of 2,400** attacked episodes (73.8%) against **315** (13.1%) for a tuned amount-threshold rule; under the liability-first preset, 1,946 (81.1%) against 462 (19.3%). The weakest position is context suppression, and the console offers it as an injectable scenario on purpose.

Independent cross-check

`analysis/crosscheck.py` reproduces the fit in scikit-learn and SciPy from the exported feature stream: the per-signal weights agree to five decimal places, and sklearn's isotonic regression reaches the same calibration error. This is what found the session-2 calibration bug (\$13). Ten of ten checks pass.

5. Data — the corpus, and what it is not

There is no public dataset of UPI payments labelled for APP fraud with session context. The nearest public material is PaySim, a mobile-money simulation with labels and amounts but no call state, no device, no entry method, and almost no repeat originators. So the primary corpus is generated, and the generator is treated as part of the model.

Fraud is not drawn from one distribution. Eight named typologies — digital arrest, KYC update, investment/trading, job/task, refund/reversal, romance, marketplace purchase, impersonation of a known person — each carry a written fingerprint and their own amount distribution, call and screen-share probabilities, urgency profile, entry-method mix, structuring propensity, hour-of-day weights, episode length and beneficiary reuse rate. The corpus contains the hard cases as well as the easy ones, in shares that reflect published fraud reporting.

Four leakage controls

1. **Chronological split.** The model is fitted on the past and judged on a future containing mule networks it never saw.
2. **Profiles advance after scoring, never before.**
3. **Beneficiary intelligence on a delayed timeline.** An account is confirmed as a mule only after an earlier victim reports and an investigation links it.
4. **The single-signal guard.** If any signal's standalone held-out AUC exceeds 0.98, the seed script warns and the CI gate fails. On generated data that is how a leak presents. This guard caught three generator artefacts during the build.

Every figure in §4 is a measurement on synthetic data. The schema is modelled on a real UPI push payment — VPAs, PSP handles, paise amounts, device and SIM hashes, session context — and the values are generated. No figure in this document is a claim about live UPI traffic, and the model card says so wherever a number depends on an assumption.

6. Simulation, fitting and evaluation — packages/sim

SCRIPT	WHAT IT DOES	WRITES
<code>npm run seed</code>	Generate the corpus, replay, fit, export the console slice with warm profile snapshots	<code>model.json</code> , <code>fit.json</code> , <code>public/data/slice.json</code>
<code>npm run evaluate</code>	Held-out replay with ground truth; portfolio report	<code>metrics.json</code> , <code>portfolio.json</code>
<code>npm run adversarial</code>	Six attacks × three defences at matched friction, with ablations	<code>adversarial.json</code>

SCRIPT	WHAT IT DOES	WRITES
<code>npm run gate</code>	Sixteen checks on a reduced corpus, as CI runs them	exit code
<code>npm run modelcard</code>	Render <code>docs/MODEL_CARD.md</code> from the artefacts	markdown
<code>npm run ingest</code>	Replay any dataset with full metrics (§7)	optional JSON
<code>npm run paysim</code>	Cross-validate on the Kaggle file if present	—

The full artefact run takes about six minutes; `--quick` uses a 1,200-payer, 21-day corpus and is what the CI gate runs. The warm profile snapshots shipped with the console slice are what make the browser's decisions match the full replay: the seed script replays to the window start and trims the snapshot to what the signals can read, about 760 KB gzipped.

7. Bringing in outside data — packages/ingest

The console shipped able to replay only its own corpus. `@kreaton/ingest` reads a delimited or JSON file, matches its columns against the twenty-eight fields the engine can use, coerces the values and builds an ordered run of payments. The same code backs the `/data` screen, `POST /api/v1/import` and the `npm run ingest` CLI, so a file behaves identically wherever it is submitted.

CONCERN	HOW IT IS HANDLED
Delimiter	Sniffed by <i>consistency</i> across lines, not frequency, so a narration column full of commas does not fool it.
Amounts	Indian and Western grouping, ₹ / INR / Rs. , trailing Dr/Cr, accounting parentheses. Sign dropped — direction is a separate field.
Timestamps	Epoch ms and s, ISO 8601, dd/mm, mm/dd, PaySim step-hours. Ambiguous order is <i>stated</i> , not picked silently; overridable per column.
No payer column	The file is read as one account holder — which is what a bank statement is.
No beneficiary id	The narration folded to a stable slug becomes the identity, so three spellings of one counterparty are one beneficiary.
Credits	Money in seeds the balance and is not scored. PaySim's PAYMENT and DEBIT are money leaving; only CASH_IN arrives.
Ordering	Payments sorted by time, stably, before scoring. A reversed file would otherwise be judged against a future it has not had.
Running balance	If present, the drain-ratio signal divides by the measurement rather than the spend-based estimate — a small core change with tests.
Mule markings	Released at the row's timestamp, never retroactively.

The report is the feature. Almost no real file carries UPI session context, and without it the engine is in the suppressed-indicator position from the adversarial suite — its weakest. Every route says so beside the figure, names each field left at its quiet value, each harmless substitution, rows dropped by reason, unreadable cells by column, and structural limits of the dataset (beneficiaries that never repeat, payers with no history). An unlabelled file gets no accuracy figure at all.

8. The console — apps/web

Framework Next.js 16.3.4, App Router, React 19.2.8, TypeScript 5.7

Styling	Plain CSS with custom properties in one <code>globals.css</code> . No Tailwind, no component library.
Charts	Hand-drawn SVG: the risk ribbon, the trade-off frontier, the policy surface, the recovery curve.
State	One engine instance per tab, held outside React and exposed through <code>useSyncExternalStore</code> , because the engine mutates its store in place and the feed advances many times a second.
Fonts	Instrument Sans and IBM Plex Mono via <code>next/font</code> .
Routes	<code>/ console</code> · <code>/data</code> your own data · <code>/policy</code> · <code>/trace</code> and <code>/trace/[txnId]</code> · <code>/audit</code> · <code>/adversarial</code> · <code>/portfolio</code> · <code>/model</code>

Design decisions recorded in `docs/PROGRESS.md` : the approve hue moved from green to teal because green against the block red measured a deutan colour-difference of 7.6, the weakest pair in the palette and the most important distinction in the product; every figure animates only on data change and never on scroll; `prefers-reduced-motion` is honoured; every route is verified free of horizontal overflow at 390px.

9. The API

GET /api/v1/health	model and policy digests, ledger head, persistence status
GET /api/v1/authorize	a complete example request body
POST /api/v1/authorize	authorise one payment
POST /api/v1/authorize/batch	authorise an ordered run, state carrying forward
POST /api/v1/import	match a file's columns and report; optionally replay it

The authorisation response carries the decision, the calibrated probability, reason codes, the hold opened (if any) with the challenge to present, the full assessment, and the sealed ledger position. Invalid bodies return every problem at once rather than the first.

The batch endpoint is *not* a loop around the single one: the structuring window, the hold linkage and every behavioural baseline carry forward between payments, so a run scored together gives different and correct answers. `/api/v1/import` replays on a scratch engine so a file sent to be understood leaves nothing behind in the live one.

What the API is and is not. It is the integration surface a payment service provider would call in its authorisation path. It is not connected to NPCI, to any switch, or to any provider sandbox. Nothing moves money.

10. Persistence — where state lives

`Store` in `packages/core/src/store.ts` is a synchronous interface, and that is a decision. Twelve signals each making a database round trip would spend a single-digit-millisecond budget several times over. Production fraud engines hold hot profile state in memory for exactly this reason. Durability is layered *behind* the working set:

```
payment → [ in-memory Store ] → decision returned    (microseconds)
                        |
                        → PersistenceSink → durable backing (afterwards)
```

`PersistenceSink` receives writes after the decision has already been returned. Its contract is one sentence: an implementation must never throw into the caller.

	SURVIVES RELOAD	SURVIVES INSTANCE RECYCLING	VISIBLE TO OTHERS
Console session (browser)	—	n/a	no
Imported dataset (IndexedDB)	yes	n/a	no
Server engine state (memory)	n/a	—	no
Audit ledger in Google Sheets	yes	yes	yes

Google Sheets as the durable backing. A service account signs a JWT with `node:crypto` and exchanges it for an access token; rows are appended with `valueInputOption=RAW` so a 64-digit hash is not reinterpreted as a number. Writes are batched because Google allows about sixty per minute; a replay at sixty payments a second would exhaust that in one second. A 429 or 5xx keeps its rows for the next flush; a 403 — a sheet not shared with the service account — lets them go rather than queueing behind an error that will never clear. Profiles are deliberately not written. On Vercel, the flush is handed to `waitUntil` so the instance stays alive until the write settles without delaying the response. Verified end to end: a decision made against the live site was read back out of the spreadsheet.

Both backings are optional. Unset, the engine behaves exactly as before and `/api/v1/health` reports `backing: memory`.

11. Testing and the quality gate

PACKAGE	TESTS	WHAT THEY PIN
core	51	Hash chain and tamper detection, cost arithmetic and boundaries, recoverability monotonicity, the observed-balance path of the drain signal
ingest	31	Delimiter sniffing, quoted fields, amount and timestamp coercion, mapping detection, statement handling, ordering, PaySim semantics, the honesty warnings
sheets	25	JWT claims, token caching, tab creation, RAW appends, batching, the retry split, the memory bound, truncation, and that no failure reaches the engine — all against a fake transport
sim	—	Covered by the gate rather than unit tests

The sixteen-check gate

`npm run gate` generates a reduced corpus, fits, replays and asserts on a reduced-scale band for each: the committed model parses; generation is deterministic; IRLS converges; no signal exceeds 0.98 AUC alone; held-out ROC AUC, PR AUC, calibration error and recall at 1% FPR are within band; replay discrimination matches the fitter; false-positive and detection rates under the default policy are within band; p99 latency; the audit chain verifies, survives an export round trip, and detects tampering at the altered record; the recoverability curve is monotone. It runs on every push and takes about a minute.

12. Deployment — GitHub Actions to Vercel

Production is deployed by GitHub Actions, not by Vercel's git integration, which is disabled in `apps/web/vercel.json`. This keeps the deploy in the same place as the tests: nothing reaches production that did not pass them, and the workflow file is the single description of how.

`ci.yml` — on every push and pull request

1. **verify:** `npm ci`, typecheck (`tsc -b` on the packages, then `next typegen && tsc --noEmit` for the app), lint, unit tests.
2. **gate:** build the packages, run the sixteen checks.
3. **build:** production `next build`, after verify.

`deploy.yml` — on push to main

1. **preflight:** publish whether `VERCEL_TOKEN` is set. A job cannot test a secret in its own `if:`, so a preceding job outputs the answer. The org's submission copy holds no secrets and therefore skips itself, keeping it green.
2. **verify:** the same checks as CI.

3. **deploy**: pinned Vercel CLI 59.16.0; `vercel pull`; `vercel build --prod`; `vercel deploy --prebuilt --prod`; then an authenticated smoke test of `/api/v1/health` on the unique deployment URL through `vercel curl`, which carries the token past deployment protection.

Project	<code>kreaton-upi</code> , Root Directory <code>apps/web</code> , Node 24
Domain	<code>kreaton-upi.vercel.app</code> , added explicitly after a rename (a rename does not re-alias)
Secrets	<code>VERCEL_TOKEN</code> (sensitive), <code>VERCEL_ORG_ID</code> , <code>VERCEL_PROJECT_ID</code>
Env vars	<code>KREATON_SHEETS_ID</code> , <code>KREATON_SERVICE_ACCOUNT_JSON</code> (sensitive), <code>KREATON_SHEETS_FLUSH_MS</code> — production only
Three copies	working copy → <code>junekashif/kreaton</code> (deploy source) → <code>Cognitia-IEM/KREATON</code> (submission), fast-forward only

13. Things that went wrong, and what they taught

Every one of these is recorded in `docs/PROGRESS.md` with how it was found, because the finding is the evidence.

The calibration bug

The isotonic fitter's pool-adjacent-violators step never merged equal levels, so the step function kept every legitimate row as its own zero block and the knot compression described the whole high-risk tail with two knots. The committed calibration was close to a straight line. Found by the Python cross-check: `sklearn` reached an ECE of 0.0018 against our 0.0144. Fixing it moved the operating point from 3.14% to 0.145% false positives at the same detection — the "FPR too high" open question from session 1 was this bug.

CI had failed on every commit from the first

The root layout used a type `Next` generates into `.next/types`. Local checks passed because `.next` was always present; a clean checkout has none. Found only because the push was watched.

The smoke test needed three fixes

`vercel curl` takes the URL positionally; `vercel deploy` prints JSON, not a bare URL; and `vercel curl` forwards unknown flags to curl, so `--token` broke it. Three deployments were READY while the test was still red. Each was findable only by running it.

The deploy that failed after adding two packages

Vercel's build command runs inside `apps/web` and was `next build` alone; the two new workspace packages resolve through `dist/`, which a fresh checkout does not have, and `transpilePackages` does not rescue a package whose entry file is missing. CI had passed because the root build compiles the packages first. Reproduced by deleting every `dist/` and building from `apps/web`; fixed by making that script compile the packages first.

The fake transport hid a bug

The Sheets sink passed 21 tests, then wrote four empty columns on every real decision. `LedgerEntry` is a discriminated union and an assessment's payment, decision and probability sit under `assessment`, not at the top level; the test fixture had invented a flat shape the engine never emits. A fake transport verifies the transport; it cannot verify an assumption about the data. Fixtures are built from the real union now.

The frozen instance

With Sheets configured on production, every flush failed with "socket disconnected before secure TLS connection was established". The sink flushes on a timer; the timer fires after the response has gone out; on Vercel that is after the instance has been frozen. `waitUntil` keeps it alive until the flush settles. The routes call it; the sink itself stays platform-agnostic.

"Labelled legitimate"

The assessment panel printed that for every payment without a fraud label. A file with no ground-truth column never said a payment was legitimate; it said nothing. It reads "no ground truth" now.

14. What is not built

- **Real payment rails.** Nothing is connected to NPCI, a switch or a provider sandbox. The API is the surface a PSP would call.
- **PaySim on the real file.** The adapter is built and smoke-tested against a file in PaySim's format; the Kaggle download has not been run in this environment.
- **A production database.** The Sheets sink is the durable backing at this scale and has the property that a judge can open it. At portfolio volume the sink is the seam to replace — `PersistenceSink` is five optional methods.
- **Beneficiary intelligence from a real feed.** Onward velocity and confirmed-mule linkage are supplied inputs standing in for network-level sharing. Without such a feed the payee-graph group loses most of its power, and the import report says so whenever a file lacks it.

Built by Team Kreation for COGNITIA 2026, problem statement FINTECH-PS2. Every figure is reproducible from the seed with the commands in §6; every claim about a defect can be checked against `docs/PROGRESS.md` and the commit history.